

OSSP (Operating Systems And Systems Programming) - 25CS2104E

Roll No. : 2520030029

Name : S. Pujith Pavan Kumar

SEC-8

SKILL-1:

Aim: To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile

To Analyze process abstraction, execute fork(), understand exec() family, analyze parentchild relationships, inspect process tree, practice system call tracing.

```
siddinapujith07@DESKTOP-FDHCAIT:~$ cd ~
siddinapujith07@DESKTOP-FDHCAIT:~$ mkdir LinuxShellProject
siddinapujith07@DESKTOP-FDHCAIT:~$ cd LinuxShellProject
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ mkdir src include
build docs tests
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ touch Makefile README.md
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ touch src/main.c
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ ls -R
.:
Makefile  README.md  build  docs  include  src  tests

./build:

./docs:

./include:

./src:
main.c

./tests:

siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ nano src/main.c
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ nano Makefile
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ make
gcc -Wall src/main.c -o shell
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ ./shell
Linux Shell Project Started!
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ ps
  PID TTY          TIME CMD
   339 pts/0        00:00:00 bash
   633 pts/0        00:00:00 ps
```

```
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ pstree
systemd--agetty
        |--chronyd-starter--| chronyd--chronyd
        |--cron
        |--dbus-daemon
        |--init-systemd(Ub)---SessionLeader---Relay(339)---bash--pstree
++
        |--init--{init}
        |--login--bash
        |--{init-systemd(Ub)}
        |--systemd-journald
        |--systemd-logind
        |--systemd-resolved
        |--systemd-udevd
        |--unattended-upgr--{unattended-upgr}
        |--wsl-pro-service--7*[{wsl-pro-service}]
++
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ nano src/fork_demo.c
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ gcc src/fork_demo.c
-o fork_demo
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ ./fork_demo
Hello
Hello
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ nano src/exec_demo.c
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ gcc src/exec_demo.c
-o exec_demo
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ ./exec_demo
Makefile  build  exec_demo  include  src
README.md docs  fork_demo  shell    tests
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ nano src/parent_child.c
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ gcc src/parent_child.c
-o parent_child
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ ./parent_child
I am Parent Process
I am Child Process
```

```
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ strace ls
execve("/usr/bin/ls", ["ls"], 0x7ffd95dfe6e0 /* 26 vars */) = 0
brk(NULL)                               = 0x62d2e39ea000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS,
-1, 0) = 0x779fa2423000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file
or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20087, ...}) = 0
mmap(NULL, 20087, PROT_READ, MAP_PRIVATE, 3, 0) = 0x779fa241e000
..
Makefile  build  exec_demo  include  src
README.md docs  fork_demo  shell    tests
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$ strace -o trace.txt
cat sample.txt
Hello World
This is my operating systems assignment.
This file will be copied using system calls.
siddinapujith07@DESKTOP-FDHCAIT:~/LinuxShellProject$
```

```

|VT0|FF0|OPOST|ONLCR, c_cflag=B38400|CS8|CREAD, c_lflag=ISIG|IC
ANON|ECHO|ECHOE|ECHOK|IEXTEN|ECHOCTL|ECHOKE, ...}) = 0
brk(0x62d2e3a4d000) = 0x62d2e3a4d000
ioctl(1, TCGETS2, {c_iflag=ICRNL|IXON, c_oflag=NL0|CR0|TAB0|BS0
|VT0|FF0|OPOST|ONLCR, c_cflag=B38400|CS8|CREAD, c_lflag=ISIG|IC
ANON|ECHO|ECHOE|ECHOK|IEXTEN|ECHOCTL|ECHOKE, ...}) = 0
ioctl(1, TIOCGWINSZ, {ws_row=40, ws_col=63, ws_xpixel=0, ws_ypi
xel=0}) = 0
ioctl(1, TIOCGWINSZ, {ws_row=40, ws_col=63, ws_xpixel=0, ws_ypi
xel=0}) = 0
ioctl(1, TCGETS2, {c_iflag=ICRNL|IXON, c_oflag=NL0|CR0|TAB0|BS0
|VT0|FF0|OPOST|ONLCR, c_cflag=B38400|CS8|CREAD, c_lflag=ISIG|IC
ANON|ECHO|ECHOE|ECHOK|IEXTEN|ECHOCTL|ECHOKE, ...}) = 0
ioctl(1, TCGETS2, {c_iflag=ICRNL|IXON, c_oflag=NL0|CR0|TAB0|BS0
|VT0|FF0|OPOST|ONLCR, c_cflag=B38400|CS8|CREAD, c_lflag=ISIG|IC
ANON|ECHO|ECHOE|ECHOK|IEXTEN|ECHOCTL|ECHOKE, ...}) = 0
statx(AT_FDCWD, ".", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_mas
k=STATX_ALL|STATX_MNT_ID, stx_attributes=0, stx_mode=S_IFDIR|07
55, stx_size=4096, ...}) = 0
statx(AT_FDCWD, ".", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_mas
k=STATX_ALL|STATX_MNT_ID, stx_attributes=0, stx_mode=S_IFDIR|07
55, stx_size=4096, ...}) = 0
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY
) = 3
fstat(3, {st_mode=S_IFDIR|0755, st_size=4096, ...}) = 0
newfstatat(AT_FDCWD, ".", {st_mode=S_IFDIR|0755, st_size=4096,
...}, 0) = 0
getdents64(3, 0x62d2e3a1e150 /* 13 entries */, 32768) = 384
getdents64(3, 0x62d2e3a1e150 /* 0 entries */, 32768) = 0
close(3) = 0
write(1, "Makefile build exec_demo inc"...100Makefile b
uild exec_demo include shell tests
README.md docs fork_demo parent_child src
) = 100
sigaltstack({ss_sp=NULL, ss_flags=SS_DISABLE, ss_size=8192}, NU
LL) = 0
munmap(0x779fa2420000, 12288) = 0
exit_group(0) = ?
+++ exited with 0 +++

```

Code:

```
#include <stdio.h>

int main()
{
    printf("Linux Shell Project Started!\n");
    return 0;
}
```

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    fork();
    printf("Hello\n");
    return 0;
}
```

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int pid = fork();

    if(pid == 0)
        printf("I am Child Process\n");
    else
        printf("I am Parent Process\n");

    return 0;
}
```

```
#include <unistd.h>

int main()
{
    execl("/bin/ls", "ls", NULL);
    return 0;
}
```

Observation :

- A Linux virtual machine was installed and configured successfully.
- GCC was configured and used to compile C programs.
- A Git repository and basic project structure were created.
- The shell architecture and basic Makefile structure were understood.
- Process abstraction was studied using `fork()`.
- The `exec()` family was used to replace a process with a new program.
- Parent-child relationships were observed using process IDs and the process tree.
- System calls generated by processes were inspected using tracing tools.

Result :

The Linux development environment was successfully prepared with GCC, Git, project structure, and Makefile. Process management concepts were demonstrated using `fork()` and `exec()`, parent-child relationships and process trees were analyzed, and system call tracing was successfully performed to understand how processes interact with the Linux kernel.